B-Side: Binary-Level Static System Call Identification

Gaspard Thévenon
ENS Lyon
Lyon, France

Kevin Nguetchouang
Grenoble INP
Grenoble, France

Kahina Lazri
Orange Innovation
Paris, France

Alain Tchana
Grenoble INP
Grenoble, France

Pierre Olivier
The University of Manchester
Manchester, United Kingdom

ABSTRACT

System call filtering is widely used to secure programs in multi-tenant environments, and to sandbox applications in modern desktop software deployment and package management systems. Filtering rules are hard to write and maintain manually, hence generating them automatically is essential. To that aim, analysis tools able to identify every system call that can legitimately be invoked by a program are needed. Existing static analysis works lack precision because of a high number of false positives, and/or assume the availability of program/libraries source code – something unrealistic in many scenarios such as cloud production environments.

We present B-Side, a static binary analysis tool able to identify a superset of the system calls that an x86-64 static/dynamic executable may invoke at runtime. B-Side assumes no access to program/libraries sources, and shows a good degree of precision by leveraging symbolic execution, combined with a heuristic to detect system call wrappers, which represent an important source of precision loss in existing works. B-Side also allows to statically detect phases of execution in a program in which different filtering rules can be applied. We validate B-Side and demonstrate its higher precision compared to state-of-the-art works: over a set of popular applications, B-Side's average F_1 score is 0.81, vs. 0.31 and 0.53 for competitors. Over 557 static and dynamically-compiled binaries taken from the Debian repositories, B-Side identifies an average of 43 system calls, vs. 271 and 95 for two state-of-the art competitors. We further evaluate the strictness of the phase-based filtering policies that can be obtained with B-Side.

CCS CONCEPTS

• **Software and its engineering → Automated static analysis**;
• **Security and privacy → Operating systems security**.

KEYWORDS

Static analysis, Binary analysis, System call filtering

MIDDLEWARE '24, December 2–6, 2024, Hong Kong, Hong Kong

ACM ISBN 979-8-4007-0623-3/24/12
https://doi.org/10.1145/3652892.3700761

ACM Reference Format:
Gaspard Thévenon, Kevin Nguetchouang, Kahina Lazri, Alain Tchana, and Pierre Olivier. 2024. B-Side: Binary-Level Static System Call Identification. In *24th International Middleware Conference (MIDDLEWARE '24), December 2–6, 2024, Hong Kong, Hong Kong*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3652892.3700761

1 INTRODUCTION

System call filtering [10] is widely used to sandbox programs in various environments such as containers [20], desktops [13, 15] and mobile [27] systems. The current methods to define filtering rules are suboptimal. On the one hand, creating generic rules that work for most applications trades off security for generality and leaves a large attack surface: for example, Android's filtering rule applies to all applications but only blocks 17 of the 271 arm64 Linux system calls [27], and Docker blocks by default only 43 of the 350+ x86-64 system calls [16]. Similarly, modern desktop software deployment and package management systems relying on system call filtering to sandbox applications fail to provide strict generic policies: Flatpak's filtering rule allows completely unrestricted access to 308 system calls for x86-64 [15], and the Firejail [14] sandbox used by AppImage only precludes 15 system calls by default [13]. Service (e.g. cloud) providers may create system call filtering policies on the basis of their needs and present them as a take-it-or-leave-it basis to clients, but these policies are either too generic to bring strong safety guarantees, or too strict to be applicable to a broad set of applications. On the other hand, manually creating filtering rules that are tailored in a per-application fashion [19, 33, 36, 39, 44] is highly secure but also very difficult to achieve and hard to maintain [32, 43], as it requires detailed knowledge about the application and can only be achieved by an expert such as the application's developer.

The solution to create tailored rules without manual effort is to automate that process: there is a need for tools that can identify the list of all system calls that can be made by an application. This task is made hard by the fact that in many environments (e.g. cloud), user applications are only available in a binary form and a cloud provider or desktop sandbox wishing to enforce a system call filtering policy cannot assume access to the sources of applications or libraries [37]. Dynamic analysis [28, 29, 34, 51, 54] is a poor fit to build per-application system call allowed/denied lists, due to the unavoidable risk of false negatives stemming from the difficulty to achieve full coverage. Existing tools based on static analysis either do not achieve good coverage [35, 46], lack flexibility and generality [4, 6, 7, 11, 16, 23, 35, 37, 47] by requiring access to the application/library sources and/or being restricted to a subset of

applications written in specific languages, or suffer from lack of precision due to suboptimal value tracking strategies during the system call identification process [6, 11].

We propose B-Side (for Binary-level System call Identifier), *a static binary analysis framework that aims to automatically and precisely identify all the possible system calls that can be issued by a given x86-64 statically or dynamically compiled ELF executable.* B-Side targets compiled executables: it makes no assumption on the availability of application/library sources and works independently of the language the application is written in. To that aim, B-Side disassembles the target binary and uses symbolic execution, a technique which was proven effective at reducing attack surface [55], to determine the types of system calls that can possibly be invoked at each of the program's system call sites.

B-Side tackles several challenges, the main one being comprehensive and precise system call identification. Being comprehensive means that every system call that may be invoked by the analyzed binary should be identified: any false negative (system call missed by the analysis but legitimately called at runtime) will result in a legitimate system call being flagged at runtime as illegal by a filtering rule derived from the analysis – something unacceptable in production. Being precise signifies that the set of system calls identified should be as small as possible, as false positives (system calls identified by the analysis but never invoked at runtime) limit the strictness of filtering rules derived from the analysis. B-Side tackles these challenges by using a search algorithm navigating the program's CFG backward from each system call site, and leveraging symbolic execution to determine every possible value that can be present in the ABI-defined CPU register containing the system call identifier at the time of invocation. Further, B-Side uses a heuristic to detect occurrences of system call wrappers and drives symbolic execution appropriately in such cases, avoiding the significant loss of precision of existing works [6, 11] in the presence of such wrappers, which are widely used in standard libraries and runtimes.

Recent works have presented advanced system call filtering policies: temporal system call specialization [17] installs different filters for different phases of execution of an application, and system call flow integrity enforces the ordering of certain system calls [5, 24] or a valid control flow in the program prior to a system call invocation [23]. We explore B-Side's capacity to analyze binary-only programs for generating such advanced filtering policies. This is achieved through the construction of an automaton in which states represent phases of execution of the program, and transitions between states correspond to invocation of certain system calls. For each phase B-Side identifies the list of possible system calls that may be invoked, as well as the system calls triggering transitions to other phases.

We validate B-Side by showing the absence of false negatives, and evaluate its precision (amount of false positives) against two competitors over 557 Debian 10 x86-64 ELF binaries. Over more than 200 dynamically-compiled binaries, B-Side identifies an average of 55 system calls, vs. 274 for Chestnut and 96 for SysFilter. We further evaluate the precision of the phase detection feature of our tool, demonstrating over an example that a phase-based filtering rule derived from the automaton would lead a reduction of 11 to 13% of the system call types allowed compared to a vanilla filtering rule enforced over the entire program's lifetime.

This paper makes the following contributions:

(1) We list and detail the various challenges arising in the domain of system call identification.
(2) We design, implement, validate and evaluate B-Side, a static binary analysis framework aiming at automatically identifying all the possible system calls that can be issued by a given x86-64 statically or dynamically compiled ELF executable, as well as phases of execution for such executables during which different sets of system calls may be invoked.

2 SYSTEM CALL IDENTIFICATION: BACKGROUND AND CHALLENGES

2.1 Dynamic vs. Static Analysis

Dynamic analysis can be used to address the problem of system call identification [12, 21, 26, 41, 42, 48, 49], however this class of techniques is well-known to struggle to achieve complete coverage [16], and thus suffer from false negatives: these approaches under-estimate the set of system call types that can be issued by an application at runtime. This leads to normal application behavior being classified as malicious which is unacceptable in most situations in production. As a result, several works rather rely on static analysis [6, 7, 11, 16, 47] which, when implemented properly, avoids the issue of false negatives. Still, static analysis overestimates and identifies a superset of the system calls that will be invoked at runtime by a program, it suffers from false positives. This reduces the strictness of the filtering policies that can be derived, but does not represent a blocking issue like false negatives. For that reason, B-Side relies on static analysis.

Static analysis tools work on the application binary or sources [6, 7, 11, 16, 23, 47]. Most make the assumption that program and/or library sources are available [4, 16, 18, 23, 37, 47]. We argue that access to sources is an unrealistic assumption in many situations. For example, a cloud provider executing its clients' applications in containers rarely has access to the source code of client programs and library dependencies. Further, source-level analysis does not scale to large numbers of applications as it is generally language-specific and needs to be tailored toward all the source-level ways to invoke system calls: language-specific wrappers, assembly code (potentially inline), etc. As previously stated, B-Side operates only on binaries and does not assume access to the sources of applications or libraries.

In essence, static binary analysis consists in: 1) disassembling the target binary; 2) recovering the program's Control Flow Graph (CFG); 3) identifying system call sites; and 4) for each site inferring what possible system calls can be issued at that point. Several of these steps involve addressing non-trivial challenges that are discussed below.

2.2 Disassembling

Disassembling arbitrary binaries is a hard problem [11, 50], due in particular to the difficulty to differentiate code from data in some scenarios [52], and to determine function boundaries [3]. Still, popular modern compilers such as GCC/LLVM do not mix code and data and embed by default metadata in binaries (including stripped

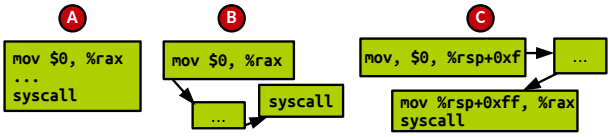

Figure 1: Scenarios in which the system call type defining immediate is: in the same basic block as the `syscall` instruction (A); in a different basic block (B); and with the immediate value propagated through memory on the stack (C).

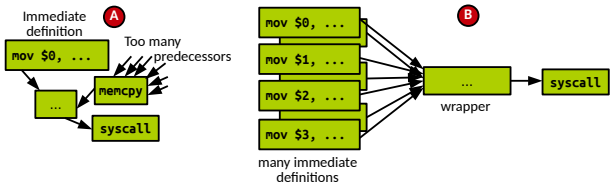

Figure 2: A function with many predecessors between the immediate definition and the `syscall` instruction make symbolic exploration difficult (A); a system call wrapper function leading to a high over-estimation of the system call set (B).

ones), such as stack unwinding information, helping in recovering function boundaries, and, as a result, modern binary analysis tools can tackle these issues for most executables.

2.3 CFG Recovery

The majority of the machine-code CFG of an executable can be recovered from disassembled code by observing direct control flow instructions taking as operand an immediate address in the code segment. The difficulty lies in resolving indirect calls, i.e. calls and jumps to a memory operand that cannot easily be determined statically [11, 45], resulting from the use of constructs such as function pointers in the sources. As a result some edges may be missing in the disassembled CFG used by the system call identification tools, which in turn may lead to false negatives due to missed system calls. This challenge is partially addressed through heuristics, that over-estimate the CFG by creating edges from each basic block ending in an indirect call toward each possible *address taken*, i.e. each address in the code segment that is used as the source of a store operation [11] (function pointer assignments). These can be complemented by other techniques including symbolic execution, backwards slicing, etc [45].

2.4 Challenges: Comprehensive and Precise System Call Identification

Once the binary is disassembled, identifying system call sites is straightforward: it consists in finding each occurrence of the `syscall` x86-64 instruction that is used to perform a system call. Understanding *what* system call(s) can be invoked at each site is more difficult and represents the key challenge tackled by B-Side.

The Linux ABI [30] states that when the `syscall` instruction is invoked, the `%rax` register should contain an integer identifying the system call to be executed. Hence, the system call identification consists in determining the possible value(s) present in `%rax` at the time each `syscall` instruction in the code segment is invoked. Due to the nature of system call invocation, in most situations a constant value (such as an immediate) will be loaded in that register, we thus can hope to determine it statically.

Naive and Use-Define Chains-based Methods. A naive method such as the one used by Angr [45] or Confine [16] does not take the CFG into account: for each occurrence of `syscall`, it considers only the containing basic block or direct predecessors and performs simple analysis to try to determine the value of `%rax` at the time of `syscall` (Figure 1 Ⓐ). Such an approach is not sufficient to identify all the system calls made in most programs because the immediate defining a system call identifier may originate from a basic block different from the one containing the `syscall` instruction, or from a direct predecessor (Figure 1 Ⓑ).

Another method considers the CFG and use-define chains [1] to track the immediate that can be placed in `%rax` [11]. This approach is also limited as it fails to track the propagation of immediates through memory. If an immediate is for example stored on the stack before being loaded into `%rax` prior to `syscall` invocation (Figure 1 Ⓒ), this method will fail to identify this value [11]. This is problematic in many cases, for example with the Go language where system calls wrappers functions, passing arguments on the stack [9] are used. As a result existing works relying on that technique require access to the sources of all the application/library code passing system call identifiers through memory [11].

Symbolic Execution. To address these issues we propose in B-Side to leverage symbolic execution [25], navigating the CFG to determine the value of all the immediates that may end up in `%rax` when `syscall` is invoked: these can be tracked even if they pass through memory. Using symbolic execution to identify system calls have been hinted at in past work, although there is no in-depth study of this technique to tackle that particular problem: although Chestnut [6] mentions it, in practice it is only used for the disassembly and not the system call identification phase, for which the tool manually tracks operands of `mov` and `xor` instructions prior to `syscall`[1].

Still, symbolic execution comes with its own set of challenges, including combinatorial explosion: when the path explored is too large, the computational complexity and/or memory footprint makes that the execution will never finish or will crash. This prevents an exhaustive forward symbolic exploration of the entire CFG from the program entry point. Hence, we perform a search starting from system call sites and going backwards towards immediate definitions.

Even with backward exploration, other problems arise: when a function called from many places in the binary (e.g. `memcpy`) is called in between the immediate definition and the corresponding `syscall` instruction, the search may lead to a state explosion because of the numerous predecessors of the popular function call site. (Figure 2 Ⓐ). With B-Side we use an advanced technique in which we search backwards in the CFG for nodes preceding the one containing `syscall`. We start *forward* symbolic execution from each

[1] https://github.com/IAIK/Chestnut/blob/91269361e53993d7e9e5acb0614559774694ba35/Binalyzer/syscalls.py#L45

predecessor, and direct the symbolic search toward the nodes identified as `syscall` predecessors during the backwards search. This allows avoiding the unneeded and costly exploration of popular functions' predecessors.

Another important issue with symbolic execution is that of system call *wrappers*, i.e. functions in the program's source code encapsulating system calls. The wrapper generally takes the system call number as parameter, followed by the system call arguments list. A good example is the `syscall()` function from the C standard library, but other examples can be found in languages such as Go and Rust, as well as in programs such as Valgrind and ptrace-based monitors. They are generally called from multiple places in a program, hence they lead to the state explosion issue described above. Further, because many (and potentially all) system call types can be made through the wrapper, a backward search from the `syscall` instruction within the wrapper up to the immediates definitions will identify a very large number of system calls: all those that can be called through the wrapper, independently of the ones actually made by the program (Figure 2 Ⓑ). In B-Side we use a heuristic to detect such wrappers and start the backward search from the wrapper invocation call sites rather than from `syscall` instruction, avoiding the state explosion and the system call overestimation issues by limiting the predecessors explored.

2.5 Soundness

None of the existing binary system call identification studies can guarantee full soundness (the absence of false negatives): for example, the identification process implemented by SysFilter [11] is based on intra-procedural analysis, leading to false negatives as the tool does not support situations in which the function declaring the constant identifying a system call is different from the function containing that system call's invocation (system call wrappers). Chestnut [6] leverages Angr's symbolic execution engine to recover the application's CFG, a technique for which full correctness is not guaranteed [2]. Similarly, we do not claim full soundness with B-Side. We detail B-Side's barriers to soundness in Section 4.6, along with arguments why it remains a useful tool in many scenarios and how it advances the state of the art on certain soundness-related challenges.

3 RELATED WORKS

Many tools have relied upon dynamic analysis [22, 29, 34, 48, 49, 51, 54]. As previously mentioned, given the limitations of this class of techniques, we rather focus on static analysis [4, 6, 11, 16, 35, 37, 46, 47]. Some of these works can afford false negatives [35, 46] because their goal is not system call filtering rule generation, hence they are unfit for our aims. Some works also assume full access to the applications' and/or library dependencies' sources [16, 18, 47]. SysPart [40] is able to automatically identify phases of execution during which different filtering rules can be applied, however it can only do so on a very small set of applications: servers software with warmup vs. serving phases.

Here we develop on 2 systems that 1) leverage static analysis, 2) aim at avoiding false negatives for security reasons and 3) work on binaries only, i.e. do not assume access to either application or library dependencies' sources.

SysFilter. SysFilter [11] focuses on precise disassembly and CFG recovery. Precise disassembly of the target binary is obtained by leveraging stack unwinding information, present even in stripped binaries. An overestimation of a precise CFG can also be recovered, including indirect calls, by using the following heuristic: the binary is first scanned to search for *addresses taken*, i.e. addresses in the code segment used as operand of a store operation (e.g. assignment of function pointers). Indirect calls in the CFG are then resolved as being made to every possible address taken. B-Side takes inspiration from this technique to recover a precise model of the CFG. SysFilter's system call identification method is quite simplistic, analyzing the machine code to determine `%rax`'s value at the time of `syscall` with use-define chains in an intra-procedural fashion. As a result the tool fails to determine system calls when the immediate defining a system call type travels through memory between functions, which is the case for example with system call wrappers, used in a non-negligible amount of binaries. This leads to SysFilter suffering from false negatives. Another limitation is that it only works on dynamically compiled binaries and does not support (non-PIC) static ones. On the contrary, B-Side uses inter-procedural symbolic execution for system call identification, which combined with our system call wrapper detection method leads to a much more robust approach that allows, as demonstrated in our evaluation, a better precision and higher compatibility with arbitrary binaries, dynamically as well as statically compiled.

Chestnut. Chestnut [6] relies on symbolic execution for disassembly only, and performs for system call identification a simple value tracking analysis on operands of `mov` and `xor` instructions (in registers only) prior to `syscall` in a backwards fashion. As we demonstrate in our evaluation (Section 5), this analysis is limited and fails in various scenarios. First, the limited scope of the exploration (30 instructions) is not sufficient to tackle many executables and libraries. Second, undirected backwards symbolic execution suffers from state explosion/lack of precision in the case of popular functions/system call wrappers. Chestnut's binary analysis includes a hardcoded detector for the Glibc's wrapper, but we confirmed that it is not able to identify system calls in other languages (such as Go) or libcs (e.g. Musl) using different wrappers. Chestnut hence relies on dynamic analysis to refine the binary analysis results, which defeats the purpose of using static analysis in the first place. B-Side uses only static analysis, is compatible with arbitrary binaries, with a heuristic able to identify system calls wrappers in arbitrary languages. Furthermore, B-Side actually relies on symbolic execution for system call identification, a technique much more robust and far-reaching compared to that of Chestnut.

4 B-SIDE: BINARY-LEVEL SYSTEM CALL IDENTIFICATION

4.1 Design's Objectives and Assumptions

We address the following research question: relying solely on static binary analysis, can we build a system call identification tool that is 1) valid and 2) precise? Validity implies the absence of false

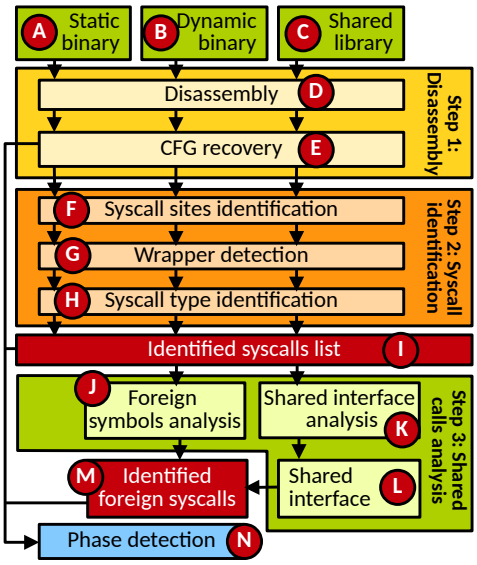

Figure 3: Overview of B-Side's system call identification process divided into 3 main steps.

negatives, i.e. system calls missed by the analysis but invoked legitimately by the program at runtime. Precision requires the minimization of false positives, i.e. system calls identified by the analysis, but that will never be invoked at runtime.

In that context we make the following assumptions. We assume no access to sources. The target programs can be written in arbitrary languages, built with arbitrary compilers, and use arbitrary standard libraries (e.g. libc, Go, Rust) possibly implementing various wrappers encapsulating system call invocations. We do not rely on any kind of dynamic analysis whatsoever. Regarding the disassembling process, we assume that the target binaries are not obfuscated. Although there is probably a certain amount of proprietary software running obfuscated e.g. in the cloud, we presume that the majority of workloads do not include obfuscated binaries, as such techniques generally impact performance negatively. We also assume that the disassembler used can correctly identify all the code as well as determine function boundaries. We do not assume that the disassembler can resolve all indirect jumps. We focus on compiled programs and scope out interpreters/managed runtime (e.g. Python, the JVM, etc.) which should be handled very differently. We rely on symbolic execution, hence the analysis may be slow or even never terminate due to the well-known path explosion issue. Although we employ optimizations to avoid these problems, we cannot eliminate them completely, and we assume it is acceptable for B-Side to time out on a subset of programs.

4.2 Overview

Figure 3 presents an overview of the analysis flow. The tool takes as input static Ⓐ and dynamically compiled executables Ⓑ with their shared library dependencies Ⓒ. In the first step of the process, binaries are disassembled. A first basic disassembly phase Ⓓ outputs a CFG of machine code basic blocks. For the vast majority of programs/libraries this CFG is incomplete as indirect jumps are not fully resolved, hence we use a heuristic to conservatively create new edges in the CFG Ⓔ to ensure that the next steps of the process will be able to explore all the possible paths in the program/library. The second step of the process regards system call identification. The system call sites are identified by scanning basic blocks for the `syscall` x86-64 instructions Ⓕ. Next, for each call site, we use a heuristic to determine if it corresponds to a system call wrapper Ⓖ. Finally, for each system call site, we proceed to identify what are the types of system calls that can be invoked at that site Ⓗ. To that aim we leverage symbolic execution and use a strategy that slightly differs according to the wrapper/non-wrapper status of the call site in question. We then obtain the list of system calls that may be invoked by the program/library Ⓘ. For dynamically compiled programs, a last step consists in determining the system calls invoked as part of shared library calls. To do so, we analyze the functions exposed by the library dependencies of the executable Ⓚ and list, for each library, the possible system calls invoked by each exposed function. This metadata is written in a file created for each library that we name the *shared interface* Ⓛ. We then list the shared library calls made by the dynamically compiled program Ⓙ, and, with the help of the shared interface, we can then output the list of system calls made by the program through shared libraries Ⓜ. Finally, the precise CFG is combined with the list of identified system calls at each site to detect phases in the program's execution Ⓝ: at runtime, a different system call allowed list can be enforced during each phase.

B-Side is implemented in Python and consists in 3217 lines of code. In the rest of this section, we detail each step of the system call identification process.

4.3 Step 1: Disassembly

Disassembly. Taking the target binary/library as input, B-Side starts by disassembling it (Ⓓ in Figure 3) using Angr [45]. Angr uses the Capstone [8] disassembly engine which is quite robust [38] and allows in particular to separate code from data and to identify function boundaries, addressing the *proper disassembly* challenge listed in Section 2. The output of that step is the machine code composing the program/library, organized into basic blocks that are linked into a CFG. A well-known limitation of modern disassembly tools is that they struggle with indirect call resolution, leading to missing edges in the CFG. We address that issue in the next step of the disassembly phase: CFG recovery.

CFG Recovery. The next step is to recover the CFG (Ⓔ in Figure 3). Although Angr/Capstone makes some attempts at resolving indirect jumps targets [38, 45], we observe that the result is not perfect. Concretely, many edges corresponding to function pointers calls are missing in the CFG of most programs/libraries. Many programs and libraries (e.g. the C standard library, used by the majority of executables) make extensive use of function pointers, hence it is important for B-Side to tackle this challenge if we wish for it to be applicable beyond a small set of restricted programs/libraries that do not use function pointers.

We thus rely on a heuristic to resolve indirect calls. We take inspiration from SysFilter [11], which defines the concept of `address taken`. An address taken is an absolute address identified in the disassembled code as located within the code segment at runtime,

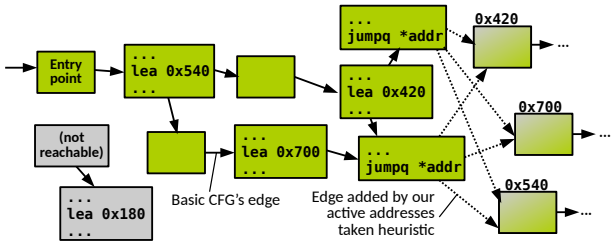

Figure 4: We use active addresses taken, i.e. addresses operands of `lea` instructions reachable from the program's entry point, to overestimate the list of indirect jump targets.

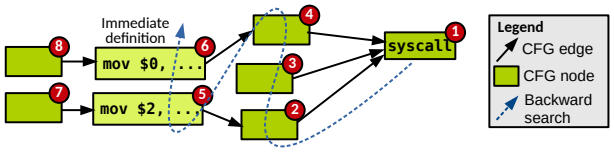

Figure 5: System call identification process: starting from a call site, predecessors are selected in BFS mode one after the other as the start node of a forward symbolic execution search. This process continues until all reachable nodes for which the symbolic execution is able to determine the system call type have been found.

which is used as the operand of a load operation. This indicates the assignment of a function pointer. In x86-64 this will concretely appear in the form of a Load Effective Address (`lea <address taken>`) instruction. The key idea is to resolve the possible targets of each indirect jump to be the list of all addresses taken in the binary. This represents a conservative overestimation of the locations that can be reached from a basic block ending with an indirect jump instruction.

Although the number of addresses taken is generally relatively low (a few tens of addresses in most binaries), we observe that it still gives a relatively large overestimation of the CFG, translating into an overestimation of the system calls identified. To tackle that issue, we refine the notion of address taken and introduce the concept of *active addresses taken*. The set of active addresses taken in a program/library represents the subset of addresses taken that are loaded with `lea` in CFG nodes that are reachable from the program's entry point or from the library's exposed functions invoked by a given program. Identifying active addresses taken is an iterative process. For a program, we start with the basic CFG obtained from Angr/Capstone. We then identify a first set of active addresses taken starting from the entry point. Next we update the CFG's indirect calls with this first set of addresses taken, and restart the process from the entry point as new active addresses taken may be discoverable given the new edges added to the CFG. This process, illustrated in Figure 4, continues until no new active addresses taken are found and the precise CFG is obtained. For a library, we first identify the functions exposed by a library that are invoked by a given dynamically compiled executable. To that aim we list the library's functions that are reachable from the program's entry point once we have obtained its precise CFG. We then repeat the process of finding active addresses taken within the library, by considering as entry points each of the exposed functions reachable by the considered program. This tackles the aforementioned challenge regarding precise CFG recovery.

4.4 Step 2: System Call Sites and Type Identification

After disassembling and constructing a precise CFG, the next step consists in identifying all the system call sites locations (Ⓕ in Figure 3). Assuming a precise disassembly, this is a straightforward operation, consisting in finding all occurrences of the `syscall` instruction. At that point we assume that the CFG obtained from the previous step is correct and consider only the occurrences that are reachable from the program's entry point (or exposed function for shared libraries).

The next step is to identify, for each considered `syscall` occurrence, the possible system call type(s) that can be made at that particular point. That process (Ⓗ in Figure 3) is slightly different depending on the `syscall` occurrence being part of a system call wrapper or not (Ⓖ). Here we describe how that is achieved when the call site is not part of a wrapper. Wrappers will be discussed next.

Identifying the system call type(s) that can be invoked by a `syscall` occurrence that is not a wrapper is illustrated in Figure 5. For each system call site (e.g. ① in Figure 5), B-Side selects predecessor nodes in a Breadth-First Search (BFS) manner, in our example starting with ②. For each of these predecessors p, B-Side starts a forward symbolic execution process from p to the system call site. In order to speed up the symbolic execution process and avoid the search getting lost in paths not leading to the system call site, we use directed symbolic execution and aim to reach the nodes leading to the `syscall` instruction, i.e. the nodes we have already identified in the BFS backwards search.

Once the system call site is reached by the symbolic search, we query the value in `%rax`. If that value is *symbolic*, as will be the case when we start from ②, ③ or ④, the symbolic search was not able to determine a concrete value for `%rax`'s content. In such case B-Side continues with the backwards BFS process. The symbolic search will succeed once it starts from a node which defines an immediate that will end up in `%rax` at the call site. It is the case for nodes ⑤ and ⑥ in our example in Figure 5. The backwards search process stops for a given path once such an immediate-defining node has been found. For example once the symbolic search starting from ⑥ is successful, the backwards search does not explore ⑧. Similarly, we do not explore ⑦ because ⑤ is immediate-defining. Once all such immediate-defining nodes directly reachable from the call site have been found, we can guarantee that we determined the value of `%rax` for every execution path reaching the call site. We output the list of all possible system calls that can be made at that site: in our example that would be 0 (`read`) and 2 (`open`).

System Call Wrapper Detection Heuristic. As mentioned above, detecting system call wrappers is important for a comprehensive analysis that aims to scale to arbitrary binaries compiled from different languages, using different standard C libraries, etc.

B-Side uses a heuristic in order to identify system call wrappers in the machine code (step Ⓖ in Figure 3). This heuristic works as follows. Recall that Capstone/Angr is able to infer function boundaries from the disassembled machine code. The key idea behind the heuristic is that we want to know whether the system call number is necessarily determined between the start a function containing a system call site, and the site in question. If it is, we conclude that the function is not a wrapper. Otherwise, then this system call number is determined elsewhere, for example by an argument of the function, and we conclude that it is a wrapper. Based on these considerations, in order to detect wrappers, B-Side employs a two-phases analysis that aims to minimize reliance on computationally-expensive symbolic execution.

The first phase is a simple use-define chain analysis that is fast but may yield false positives. Starting from the system call site, we search backwards for `mov` instructions, up to the beginning of the function. Then, by analyzing those instructions, B-Side finds out whether `%rax` can be determined, potentially by analyzing recursively other registers (if we encounter memory operands, we consider it to be undetermined). This method can yield false positives, so if this first phase is positive, the function *may* be a wrapper, and we confirm/disprove that hypothesis with a second phase leveraging symbolic execution. B-Side launches symbolic execution between the beginning of the function and the system call site, and if at this point, `%rax` is symbolic, the function is definitively qualified as a wrapper. In such case, the symbolic search also records which of the wrapper function parameter holds the system call identifier: it is either a register or a stack slot according to the programming language function call ABI. The first phase is fast, whereas the second phase is potentially time-consuming. This is why B-Side performs symbolic analysis only if the first phase is positive.

Once a wrapper is detected, the system call identification process (Ⓗ in Figure 3) for that particular system call site is slightly different from that of non-wrapper call sites. To find the system calls made by the program through a wrapper, we start a similar search process as described above, with the difference that 1) the target code address we direct the symbolic execution toward is not the system call site but the first instruction of the wrapper; and 2) we aim to determine the value present in the register/stack slot holding the wrapper's parameter corresponding to the system call identifier, and not `%rax`.

4.5 Step 3: Dynamic Executables and Shared Libraries Processing

Decoupled Executable/Library Analysis. As the majority of executables are dynamically compiled, it is necessary for B-Side to be able to handle them. Most of the symbolic methods that we use for static binaries can be reused to analyze shared libraries. One of our goals when handling dynamically compiled executables and their shared library dependencies is to factorize the analysis of shared libraries. Indeed, many libraries are used by multiple programs (e.g. `libc.so`, `libpthread.so`, etc.). In order to avoid repeating, for each program using a shared library, some costly operations involved in processing the library, B-Side implements a two-phase approach. The first and computationally-expensive phase is done only once per library. It includes the disassembly/CFG recovery, and system call sites location/wrapper detection/type identification for each function exposed by the library, i.e. steps Ⓓ to Ⓗ in Figure 3. The second phase is done during the analysis of the main dynamically compiled executable (step 3 in Figure 3). During this phase, we load (recursively) every needed shared library, we compute (recursively) the reachable functions for every library, complete the over-approximation of each call-graph by computing active address taken and resolving indirect jumps, and deduce the system calls potentially invoked by every function of those shared libraries that may be called by the executable. This phase is executed once for each library dependency of a dynamically executable.

Library Analysis and Shared Interface. The analysis of a shared library (steps Ⓓ to Ⓗ in Figure 3) is very similar to that of an executable and consists in the following: 1) listing the functions exposed by the shared library; 2) constructing the call-graph between the different functions of the shared library, potentially by taking into account external symbols; 3) detecting `address taken` functions and indirect call sites; 4) collect system call sites, detecting wrappers and handling indirect calls and; 5) inferring system call numbers.

B-Side then outputs a result which will be referred to as the library's *shared interface* (Ⓚ in Figure 3). This interface corresponds to a set of metadata that will later be used to construct a simple and direct correspondence between the library's exported functions (e.g. `printf`, `read`, etc. for the standard libc) and the system calls each of these functions can invoke.

Concretely, the shared interface corresponds to a json file. It contains in particular 1) a function-level CFG of the library; 2) a list of system calls that can be invoked by each function; 3) the list of functions defining addresses taken and the addresses taken in question; 4) the list of functions corresponding to system call wrappers; and 5) for each function calling a function exposed by another shared libraries, the list of such calls.

Dynamically Compiled Executable Analysis. The shared interface produced once per library is used during the analysis of the main executable binary or the analyses of other shared libraries. While processing the main binary, when an external symbol is encountered (Ⓙ in Figure 3), the system calls it can trigger are determined using the shared interfaces.

During this phase, B-Side loads recursively every shared library needed by the executable, computes recursively the reachable exposed functions and active addresses taken of each library, refines the over-approximation of each call-graph with these addresses taken, and deduces the system calls that can be invoked by every exposed function of those shared libraries that may be called by the executable. The first step is to determine an ordering of the nodes compatible with this DAG, which we implemented using a priority queue. Then, following this order, B-Side computes the reachable functions and active addresses taken of every shared library, by propagating the information of the reachable functions of its predecessors. Next, following the reverse order, by resolving indirect calls with the identified active addresses taken, B-Side deduces the complete call-graph of every shared library, and constructs a simple interface associating to every function exposed by the library a set of system calls that may be invoked when the function in question is called. Then, this interface is used to construct recursively the

interfaces of the other libraries. Finally, we are able to analyze the executable, and to handle every external symbol in it.

Shared objects loaded at runtime through primitives such as `dlopen` (e.g. modules) are analyzed by B-Side similarly to shared libraries. We make the assumption that the user is able to identify the modules that can be loaded by an application's main binary: due to the nature of runtime loading mechanisms, it is very hard to identify them automatically. It is also worth noting that the use of these mechanisms is relatively rare, e.g. in the set of 557 programs from the Debian repositories we consider in evaluation, less than 10% use `dlopen`.

4.6 Soundness and Performance Barriers

As mentioned earlier, similarly to all other works on binary-only system call identification [6, 11], B-Side cannot totally guarantee soundness, i.e. the absence of false negatives. This is due in part to the use of Angr which, in line with similar reverse-engineering tools, does not fully guarantee the correctness of the recovered CFG. The complete and sound disassembly of arbitrary programs is an undecidable problem [53], and we consider it to be orthogonal to our effort. Further, please note that we successfully validate in evaluation (Section 5.1) the correctness of filtering rules derived from B-Side's analysis over a set of popular applications, with high-coverage workloads (full test suites). Another barrier to soundness in B-Side is our system call wrapper detection method, which is a heuristic in nature. Note that it has been validated over wrappers implemented in different libc types (Glibc, Musl, Bionic) and languages (Go, Rust), over different versions. Another issue related to performance is the fact that the symbolic execution search may never finish or run out of memory. We demonstrate in evaluation that the success rate (no timeout/crash) of B-Side on a large set of 500+ binaries is much higher than that of competitors.

4.7 Phase Detection

Observing that the set of system calls that can be invoked by a program may vary with its phases of execution, recent works [17] have presented the concept of temporal system call specialization, in which different filtering rules are applied to a process, one per phase of execution, for enhanced security.

Existing work uses dynamic analysis to detect phases and the different system calls that may be invoked during them. Given the previously-mentioned downsides of dynamic analysis, with B-Side we explore how efficient binary-level static analysis is at detecting such phases. An intuitive method would be to navigate the CFG and merge into phases sets of nodes that contain system call sites and that are highly connected. In practice, we observe that navigating the CFG to perform that operation is a very costly operation that does not scale well to medium/large programs. Hence, we propose below an optimized method based on automata theory.

The key idea of this method is to transform the CFG and list of possible system calls issued at every system call site into a Deterministic Finite Automaton (DFA) in which states are sets of program's instruction (program's states) and transitions are the types of system calls used to switch states: the input alphabet of the automaton is the list of all system calls that can be invoked by the program.

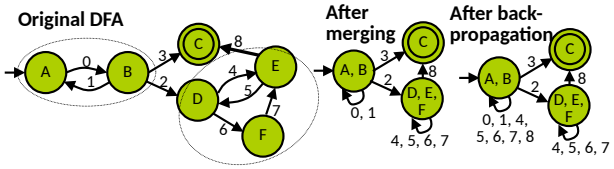

Figure 6: Automaton-based phase detection: states represent sets of basic blocks and transitions are system call invocations. Highly-connected states are merged to form phases, each with a system call allowed list (transitions from the phase).

B-Side first creates a Non-deterministic Finite Automaton (NFA), by decorating the outgoing edges of every node containing a system call site with the list of system calls that can be invoked at that site. All other edges are transformed into empty (ϵ-) transitions. At that stage the number of ϵ-transitions in the NFA is very high as in the initial CFG only a small portion of nodes include system call sites. The next step is to transform the NFA into an equivalent DFA that will be used for phase identification. To that aim B-Side uses a standard powerset construction algorithm. That process eliminates ϵ-transitions and ensures that every state has at most one outgoing transition of a given system call type.

B-Side then obtains a DFA in which highly-connected states can be merged to obtain phases as depicted in Figure 6. Each phase is characterized by the set of basic blocks composing it and a list of allowed system calls during that phase (transitions from that phase). Outgoing (non-self) transitions represent system calls transitioning to other phases. The final step consists in back-propagating authorized system calls to predecessor states. Note that it is needed only when Linux's seccomp is used as the runtime filter. Note that the example in Figure 6 is overly simplistic for the sake of illustration. In reality, the number of states is higher, and they are much more connected due to the CFG overestimation and to the NFA to DFA transformation process. This transformation process is highly optimized, and the automaton-based method is much faster than the graph navigation one: for a simple hello world static binary (32K basic blocks of machine code), it takes 41 seconds vs. 700 seconds for the intuitive CFG navigation technique presented earlier. For a medium-sized program (Nginx, 83K basic blocks), that difference is 20 minutes vs. 4 hours.

Existing works regarding phase-based filtering [17] assume access to the application sources so that an expert can manually place in the code calls to install new rules at phase boundaries. Given B-Side's assumptions (no source access), enforcing phase-based filtering would have to be achieved differently. The most efficient way would be to slightly modify the filtering mechanism in the kernel [24] (e.g. seccomp) to detect phase changes by monitoring, when a system call is invoked, the system call type and the application's return address. Another possibility would be a user-space monitor [23]. These solutions would also allow overcoming seccomp's limitation of being able to subsequently install only stricter rules, hence to drop the back-propagation step for more flexible

and secure phase-based policies. Still, we believe modifying seccomp/developing a user-space monitor falls out of the scope of this paper that focuses on the problem of system call identification.

5 EVALUATION

In this evaluation we answer the following questions:

- Is B-Side valid, does it suffer from false negatives? (§5.1)
- How precise is B-Side, i.e. to what extent does it suffer from false positives? How does this overestimation compare to state-of-the-art competitors? (§5.2)
- What are the execution times and memory footprints of B-Side's analysis? (§5.3)
- How strict would be the filtering policies that can be derived from B-Side's phase detection analysis? (§5.4)
- How efficient would be the filtering rules derived from B-Side's analysis at protecting a large set of programs against a set of real-world CVEs? (§5.5)

In several experiments we compare B-Side to Chestnut [6] and SysFilter [11]. These competitors are the most relevant because, similarly to B-Side and contrary to other works [4, 23, 37], they do not assume access to application/library sources.

5.1 Validation

Existing works validate the correctness of the filtering rules derived from their analysis by relying on a form of dynamic analysis, as other approaches either lack precision (source-level analysis) which defeats the purpose, or are simply not doable (manual analysis). Some competitors check if standard benchmarks [16] or test suites [11] succeed while enforcing the filtering rules. This does not allow to reason about false positives (system calls identified by the tools but not actually issued by the application), hence we take a different approach. We compare for a set of applications the system calls identified by B-Side to a ground truth, in order to detect false positives as well as possible false negatives (system calls present in the ground truth but not detected by the analysis). Similar to existing works [6, 11, 16], the ground truth is established using dynamic analysis: we trace with `strace` at runtime the system calls invoked by each considered application when running its entire test suite. We select 6 applications: Redis, Nginx, HAProxy, Memcached, Lighttpd and SQLite. Similarly to prior works, we assume that their test suites achieve sufficient coverage for dynamic analysis to produce a ground truth of good quality [6]. These applications are highly popular and widely deployed in production, hence they need to be extensively tested with high-coverage test suites. Note that some applications (e.g. Nginx) load modules at runtime through `dlopen`. These are processed alongside the main binary, similarly to shared libraries.

The results are on Figure 7. We observe that B-Side does not present any false negatives, while the competitors do. We believe that this is mainly due to the lack of system call wrapper handling in both Chestnut and SysFilter, which leads to system calls that will be missed by the analyses. Chestnut presents a lower amount of false negatives compared to SysFilter, which is mainly due to the very high number of system call identified (more than 250 for each application), denoting a high amount of false positives. We also

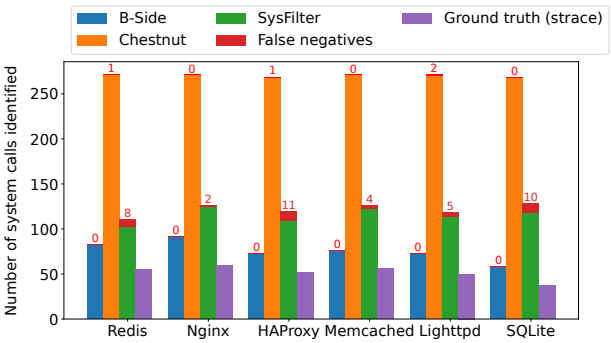

Figure 7: System calls identified by B-Side, Chestnut, SysFilter, and `strace` (on test suites) on 6 applications.

Table 1: F_1 scores for B-Side, Chestnut and SysFilter over the 6 binaries considered for validation.

	F_1 Score					
	Redis	**Nginx**	**HaProxy**	**Memcached**	**Lighttpd**	**SQLite**
B-Side	0.8	0.8	0.87	0.88	0.81	0.78
Chestnut	0.33	0.36	0.32	0.34	0.3	0.24
SysFilter	0.59	0.62	0.51	0.58	0.55	0.35

successfully validated B-Side on these applications with an alternate C library, Musl, both in statically and dynamically compiled modes.

5.2 Precision

Popular Applications. Table 1, presents the F_1 score (harmonic mean of precision and recall) for B-Side, Chestnut and SysFilter on each of the binaries considered in the validation. B-Side's score is consistently higher than the competitors': its average F_1 score over all applications is 0.81, vs. 0.31 for Chestnut and 0.53 for SysFilter. The main reason for that is the high number of false positives shown by these competitors, as illustrated on Figure 7. In terms of dangerous system calls [6], we confirmed that B-Side is able to filter out `execve` on Nginx/Memcached, and `execveat` on all popular applications.

Debian Binaries. To measure precision at scale, we execute it over a vast set of programs: we selected from the Debian 10 x86-64 repositories (`main`, `contrib` and `non-free`) every ELF executable containing at least one occurrence of the `syscall` instruction. For dynamically-compiled binaries, we also select the corresponding shared library dependencies that include this instruction. We gather a total of 557 executables: 231 static binaries and 326 dynamically-compiled with their 59 shared library dependencies. We confirmed by exploring the corresponding source packages that these binaries come from the compilation of many languages: C, C++, Haskell, Go, etc. B-Side is compared to Chestnut and SysFilter over all binaries. We compare the number of executables for which each system succeeds/fails, as well as the number of system calls identified.

The results are summarized in Table 2. On dynamic binaries, B-Side's precision is superior: it identifies on average 55 system calls vs. 274 for Chestnut and 96 for SysFilter. Figure 7 details the exact number of system calls identified by each solution for each

Table 2: B-Side vs. Chestnut and SysFilter on 557 binaries extracted from the Debian 10 repositories. For each system considered we sum up the number of static and dynamic binaries upon which the analysis succeeds to aggregate the number of successes over the total set of binaries considered, and proceeded similarly for the failures.

		B-Side	Chestnut	SysFilter
All binaries	**#Success**	**441 (79.2%)**	**310 (55.7%)**	**109 (19.6%)**
	#Failures	**116 (20.8%)**	**247 (44.3%)**	**448 (80.4%)**
	Avg. #syscalls identified	**43**	**271**	**95**
Static executables	#Success	227 (98.3%)	4 (1.7%)	1 (0.4%)
	#Failures	4 (1.7%)	227 (98.3%)	230 (99.6%)
	Avg. #syscalls identified	32	28	24
Dynamic executables	#Success	214 (65.6%)	306 (85.9%)	108 (30.3%)
	#Failures	112 (34.4%)	20 (14.1%)	218 (69.7%)
	Avg. #syscalls identified	55	274	96

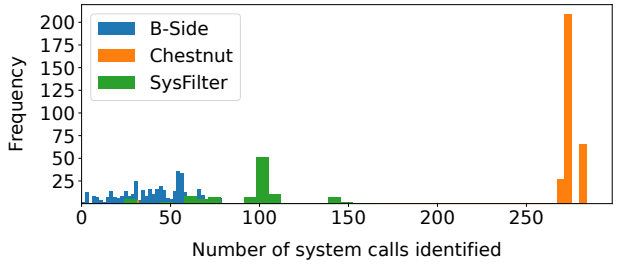

Figure 8: Distribution histogram of the number of system calls identified by B-Side, Chestnut, and SysFilter over the set of Debian binaries upon which each tool successfully runs.

of the 6 applications used in validation. Chestnut always identifies more than 268 system calls, and SysFilter more than 103. B-Side reports between 49 and 84 (58 and 92 when accounting for false negatives). These numbers are close to the ground truth established from dynamic analysis on test suites, highlighting the low number of false positives in B-Side's analysis. Chestnut and SysFilter are not as precise and show many false positives.

Figure 8 presents the distribution of the number of system calls identified for all applications in our set. For the vast majority of executables Chestnut identifies around 270 system calls, and there are very few variations of that number with applications. SysFilter also reports around 100 system calls in most cases. On the contrary, for most applications B-Side generally identifies considerably fewer system calls (between 1 and 90), and the numbers vary highly on a per-application basis. This further shows the higher precision of B-Side and its ability to identify a close superset of the system calls that may be invoked by an application at runtime.

Although Chestnut and SysFilter identify fewer system calls than B-Side in static binaries (respectively 28 and 24 vs. 32), they fail on most static binaries (respectively 227/231 and 230/231), making the comparison relatively meaningless. We investigated the reasons behind Chestnut's failure on static binaries and found that it is linked to its lack of management of system call wrappers. Concerning SysFilter, its failure is due to its lack of support for non-PIC binaries.

Table 3: B-Side's analysis execution time, memory footprint (peak resident set size), and number of basic blocks executed symbolically during the system call identification phase on the applications used in validation.

Application	Execution time				Peak RSS	BBs explored in identification phase
	CFG recovery	Wrappers identification	Syscalls identification	Total		
Redis	682 sec	128 sec	303 sec	25 min	11.9 GB	958
Nginx	407 sec	57 sec	9 sec	8 min	2.5 GB	21
HaProxy	999 sec	213 sec	98 sec	22 min	6.2 GB	42
Memcached	376 sec	45 sec	810 sec	21 min	9.8 GB	1105
Lighttpd	344 sec	39 sec	12 sec	7 min	2.4 GB	23
SQLite	218 sec	33 sec	123 sec	7 min	3.1 GB	499

Failures. We observe that B-Side fails on part (112/326) of dynamically compiled executables, due to the analysis not finishing within the maximum time window we defined (3 hours max for each program). Studying these timeouts, the vast majority of them (73%) happen during the CFG construction phase which is generally the longest one (see Table 3). For this phase we entirely rely on Angr's CFG recovery feature. As such we consider that issue orthogonal to B-Side and hope Angr will be enhanced to support faster CFG recovery for most binaries. A few timeouts (15%) happen during the core system call identification phase leveraging symbolic execution. The amount of paths symbolically explored when the analysis times out during the system call identification phase varies according to the binary, but is generally high (thousands of basic blocks explored). Giving more time to the analysis to complete would address the problem, although it is hard to determine how much time would be needed. A small amount of timeouts (12%) also happen during the execution of our wrapper identification analysis. Optimizing this aspect of B-Side will be a priority in future works. As one can observe on Table 3, the wrappers and system calls identification phases are generally shorter than the CFG recovery, which explain the lower amount of timeouts during wrapper/system call identification.

5.3 Execution Time and Memory Footprint

Table 3 presents the execution time and peak resident set size of B-Side's analysis when applied to each of the applications used in our validation (§5.1). Execution times are in the order of minutes, ranging from 6.5 minutes (SQLite) to 26 minutes (Redis). The table also presents the execution time of the three main steps of the analysis: as one can observe, the time taken by Angr to recover the CFG generally dominates. Note that the sum of the 3 main phases of the analysis (CFG recovery, wrappers identification, and system call identification) is inferior to the total analysis time as there are other steps involved (e.g. loading the binary) which execution time is not presented here. The memory footprint varies from 2.4 GB (Lighttpd) to 11.9 GB (Redis). Execution time and memory footprint are a one time cost to generate a filter for a given binary with B-Side, and as such optimizing these was not a primary objective for our work.

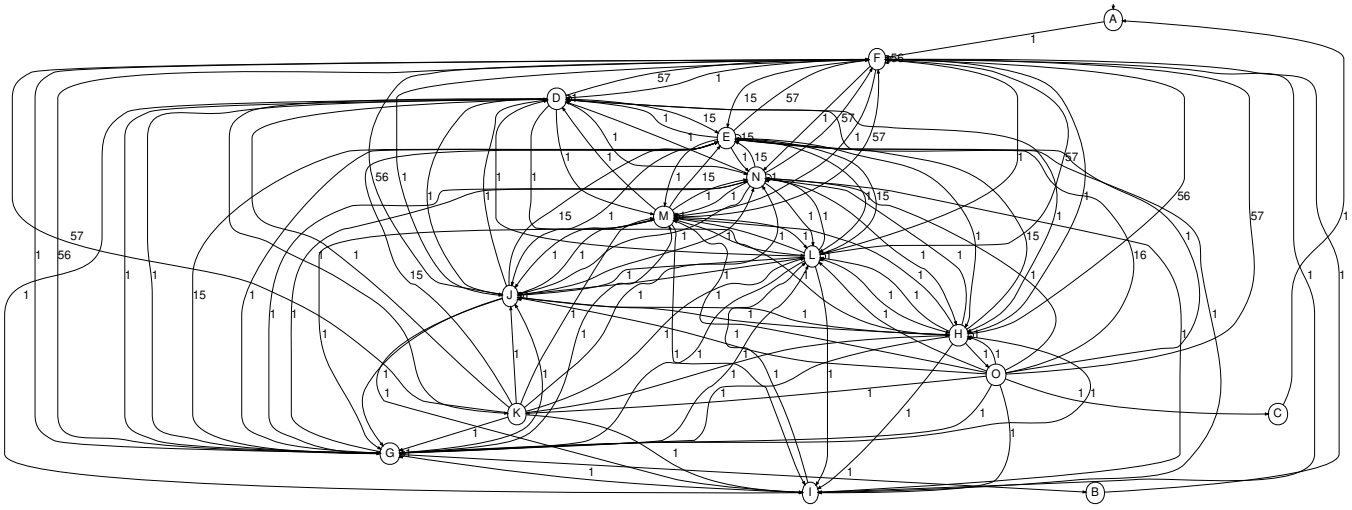

Figure 9: Automaton representing Nginx phases of execution extracted by B-Side. Note that for clarity reasons, for each state we do not have one edge per allowed system call, but rather one edge per possible destination phase, labeled with the number of different system calls types triggering that transition.

5.4 Phase Detection

To evaluate the effectiveness of B-Side's phase detection technique, we ran it on the 6 applications presented in the validation. Here we describe the results for Nginx, but note that the observations are similar for all 6 applications.

Figure 9 presents a simplified view of the automaton extracted by B-Side on Nginx (see Section 5.4) before the back-propagation step (see Section 4.7) and representing its various phases of execution. In this figure, each state corresponds to a phase. For the sake of clarity, each edge outgoing from a phase P to another phase P' represents a set of system calls allowed in P, which invocation triggers a transition to P'. The edge's label indicates the number of system calls composing the set in question. Note that P and P' can correspond to the same phase. The number of system calls allowed in each phase corresponds to the sum of the labels of each outgoing edge. As a reminder the total number of system calls identified in Nginx' binary by B-Side is 93.

To further understand the output of our phase detection analysis, we present in Table 4 a summary of the automaton: the number of detected phases and, to determine how strict each phase is, the number of identified system calls per phase. B-Side identifies 15 phases for Nginx. In this table cells give the number of system calls triggering a transition between a source (row) and destination (column) phases, each phase being a state of the automaton. The total column gives the number of system calls allowed in each phase, i.e. its strictness. To estimate how much time the program would spend in each phase, we also report in the last column the code size of each phase by summing the sizes of all basic blocks composing each phase. Note that due to the method we use to make the automaton deterministic, a basic block can belong to several phases.

We can categorize phases into two classes. Small phases (A-C and I) are very strict with a single system call allowed, but also small (a few bytes of code), hence it is likely that the program will spend the majority of its execution outside such phases. Large phases (D-H and J-O) allow a higher number of system calls: 79 to 83, representing respectively 85% and 89% of the total amount of system calls identified in the binary (93). Such phases have a much higher size (840-860 KB) and the program will spend most of its execution in them. Hence, for Nginx, using a phase-based filtering system would lead to an average increase in strictness of 11 to 15%. We observed similar numbers for the other 6 applications studied.

Table 4: Nginx phase automaton: each cell gives the size of a set of system calls allowed in the source phase and triggering a transition to the destination phase. The last columns give the total number of system calls allowed in each phase and the total size of the basic blocks composing the phase.

		Destination phase															Total	Size
		A	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**	**L**	**M**	**N**	**O**	**(/93)**	**(B)**
Source phase	**A**	-	-	-	-	-	1	-	-	-	-	-	-	-	-	-	1	48
	B	-	-	-	-	-	1	-	-	-	-	-	-	-	-	-	1	15
	C	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	1	11
	D	-	-	-	1	15	57	1	1	1	1	1	1	1	1	-	81	841k
	E	-	-	-	1	15	57	1	1	1	1	-	1	1	1	-	80	863k
	F	-	-	-	1	15	57	1	1	1	1	-	1	1	1	-	80	864k
	G	-	1	-	1	15	56	1	1	1	1	-	1	1	1	-	80	841k
	H	-	-	-	-	15	56	1	1	1	1	-	1	1	1	1	79	841k
	I	-	-	-	-	-	-	-	-	-	-	-	1	-	-	-	1	22
	J	-	-	-	1	15	56	1	1	1	1	-	1	1	1	-	79	841k
	K	-	-	-	1	15	57	1	1	1	1	-	1	1	1	-	80	840k
	L	-	-	-	1	15	57	1	1	1	1	-	1	1	1	-	80	841k
	M	-	-	-	1	15	57	1	1	1	1	-	1	1	1	-	80	841k
	N	-	-	-	1	15	57	1	1	1	1	-	1	1	1	-	80	841k
	O	-	-	1	1	16	57	1	1	1	1	1	1	1	1	-	83	841k

5.5 Security Support

We aim to understand how efficient the filtering rules that can be derived from B-Side's analysis would be at protecting against a set of real-world CVEs. We extracted from the literature [11, 16, 31] a list

Table 5: Percentage of Debian binaries protected by filtering rules derived from B-Side's analysis for a set of CVEs.

CVE	System call(s) involved	CVE type*	% of Debian programs protected
2021-35039	init_module	B	99.28%
2019-13272	ptrace	P	83.21%
2019-11815	clone, unshare	UaF	87.77%
2019-10125	io_submit	UaF	100%
2019-9857	inotify_add_watch	DoS	99.76%
2019-3901	execve	L	61.39%
2018-18281	ftruncate, mremap	UaF	90.89%
2018-14634	execve, execveat	P	100%
2018-13053	clock_nanosleep	DoS	99.28%
2018-12233	setxattr	P, L, DoS	99.28%
2018-11508	adjtimex	L	100%
2018-1068	compat_sys_setsockopt	W	100%
2017-18509	setsockopt, getsockopt	P, DoS	64.99%
2017-18344	timer_create	R	100%
2017-17712	sendto, sendmsg	P	96.40%
2017-17053	modify_ldt, clone	UaF	100%
2017-14954	waitid	B, P, L	100%
2017-11176	mq_notify	DoS	100%
2017-6001	perf_event_open	P	100%
2016-7911	ioprio_get	P, DoS	100%
2016-6198	rename	DoS	91.61%
2016-6197	rename, unlink	DoS	91.61%
2016-4998	setsockopt	P, DoS	53.96%
2016-4997	setsockopt	P, DoS	53.96%
2016-3134	setsockopt	P, DoS	53.96%
2016-2383	bpf	L	100%
2016-0728	keyctl	P, DoS	100%
2015-8543	socket	P, DoS	68.35%
2015-7613	semget, msgget, shmget	P	100%
2014-9903	sched_getattr	L	100%
2014-9529	keyctl	DoS	100%
2014-8133	set_thread_area	B	100%
2014-7970	pivot_root	DoS	99.52%
2014-5207	mount	P	72.42%
2014-4699	fork, clone, ptrace	P, DoS	84.41%
2014-3180	compat_sys_nanosleep	R	100%

*B: check bypass, L: info leak, UaF: use afer free, R/W: memory read/write primitives, DoS: denial of service, P: privilege escalation

of CVEs involving various (Linux) kernel-level vulnerabilities that can be triggered through system calls. Each CVE is mapped to the system call(s) triggering it. For a CVE C triggered by a system call S, if in a program P B-Side does not identify S, then one could derive a filtering policy for P precluding S and thus protecting against C. We consider the list of 557 Debian binaries from Section 5.2, and compute the percentage of programs from that set that would be protected against each CVE given a filtering rule derived from B-Side's analysis.

We gathered a list of 36 CVEs for which system calls are involved in the attack process. These CVEs are extracted from the following papers: SysFilter [11], Confine [16], as well as Kite [31], and filtered out CVEs prior to 2014 for space reasons. They are presented in Table 5, along with a description of their impact, and the percentage programs protected. We find that on average over all CVEs, a filtering rule derived from B-Side's analysis would protect 90.33% of the Debian binaries we consider. For 16 CVEs, 100% of the binaries are protected, and there is no CVE for which less than 54% of the programs are protected. The percentage of binaries protected depends on the popularity of the system call triggering the vulnerability. For CVEs caused by rarely-used system calls, e.g. CVE-2019-10125 relying on `io_submit` and CVE-2016-2383 relying on `bpf`, a filtering rule will protect the majority of binaries (100% for these two). With popular system calls, such as `setsockopt` from CVE-2016-4998, fewer applications are protected (54%). Still, overall, these numbers show that a high number of vulnerabilities can be avoided through filtering rules derived from B-Side's analysis.

6 CONCLUSION

System call filtering brings the need for automated binary-only system call identification techniques. B-Side is a static analysis tool identifying a superset of the system calls an executable may invoke at runtime, without assuming access to sources. It leverages symbolic execution as well as a heuristic to detect system call wrappers to provide precise results. It can also detect phases of execution in a program in which different filtering rules can be applied. B-Side offers a high precision: validated over six popular applications, B-Side's average F_1 score is 0.81, vs. 0.31 and 0.53 for competitors. Over more than two hundred dynamically-compiled binaries, B-Side identifies an average of 55 system calls, vs. 274 and 96 for competitors.

ACKNOWLEDGMENTS

We thank the anonymous reviewers and our shepherd, Emanuel Onica, for their insightful feedback and invaluable help increasing the paper's quality. This work was partly funded by Orange Innovation; the UK's EPSRC grants EP/V012134/1 (UniFaaS), EP/V000225/1 (SCorCH), EP/X015610/1 (FlexCap); and the UK Industrial Strategy Challenge Fund (ISCF) under the Digital Security by Design (DSbD) Programme delivered by UKRI as part of the projects 10017512 (MoatE) and 75243 (Soteria).

REFERENCES

[1] Alfred V Aho, Ravi Sethi, and Jeffrey D Ullman. 1986. Compilers, principles, techniques. *Addison wesley* 7, 8 (1986), 9.
[2] Angr Contributors. 2023. Angr Documentation - Control-flow Graph Recovery (CFG). https://docs.angr.io/en/latest/analyses/cfg.html.
[3] Tiffany Bao, Jonathan Burket, Maverick Woo, Rafael Turner, and David Brumley. 2014. BYTEWEIGHT: Learning to recognize functions in binary code. In *23rd USENIX Security Symposium (USENIX Security 14)*. 845–860.
[4] Alexander Bulekov, Rasoul Jahanshahi, and Manuel Egele. 2021. Saphire: Sandboxing PHP Applications with Tailored System Call Allowlists. In *30th USENIX Security Symposium (USENIX Security 21)*. 2881–2898.
[5] Claudio Canella, Sebastian Dorn, Daniel Gruss, and Michael Schwarz. 2022. SFIP: Coarse-Grained Syscall-Flow-Integrity Protection in Modern Systems. *arXiv preprint arXiv:2202.13716* (2022).
[6] Claudio Canella, Mario Werner, Daniel Gruss, and Michael Schwarz. 2021. Automating Seccomp Filter Generation for Linux Applications. In *Proceedings of the 2021 on Cloud Computing Security Workshop*. 139–151.
[7] Claudio Canella, Mario Werner, and Michael Schwarz. 2021. Enter Sandbox. Black Hat Asia 2021. https://www.blackhat.com/asia-21/briefings/schedule/index.html.
[8] Capstone Contributors. 2022. Capstone Disassembler Website. https://www.capstone-engine.org/.
[9] Austin Clements. 2020. Proposal: Register-based Go calling convention. https://go.googlesource.com/proposal/+/refs/changes/78/248178/1/design/40724-register-calling.md.
[10] Jonathan Corbet. 2009. Seccomp and sandboxing. *LWN. net, May* 25 (2009).
[11] Nicholas DeMarinis, Kent Williams-King, Di Jin, Rodrigo Fonseca, and Vasileios P Kemerlis. 2020. Sysfilter: Automated system call filtering for commodity software. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID) 2020*. 459–474.
[12] Docker-slim Contributors. 2021. Docker-slim. https://github.com/docker-slim/docker-slim.
[13] Firejail Contributors. 2022. Firejail Seccomp Filtering Rule. https://github.com/netblue30/firejail/blob/master/src/include/seccomp.h.

[14] Firejail Contributors. 2022. Firejail Security Sandbox. https://firejail.wordpress.com/.

[15] Flatpak Contributors. 2022. Flatpak Seccomp Filtering Rule. https://github.com/flatpak/flatpak/blob/main/data/flatpak-docker-seccomp.json.

[16] Seyedhamed Ghavamnia, Tapti Palit, Azzedine Benameur, and Michalis Polychronakis. 2020. Confine: Automated system call policy generation for container attack surface reduction. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID) 2020)*. 443–458.

[17] Seyedhamed Ghavamnia, Tapti Palit, Shachee Mishra, and Michalis Polychronakis. 2020. Temporal system call specialization for attack surface reduction. In *29th USENIX Security Symposium (USENIX Security 20)*. 1749–1766.

[18] Seyedhamed Ghavamnia, Tapti Palit, and Michalis Polychronakis. 2022. C2C: Fine-grained Configuration-driven System Call Filtering. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (Los Angeles, CA, USA) *(CCS '22)*. Association for Computing Machinery, New York, NY, USA, 1243–1257. https://doi.org/10.1145/3548606.3559366

[19] Google. 2022. Chromium Documentation: Linux Sandboxing. https://chromium.googlesource.com/chromium/src/+/0e94f26e8/docs/linux_sandboxing.md.

[20] Google. 2023. What is gVisor? https://gvisor.dev/docs/.

[21] Zhongshu Gu, Brendan Saltaformaggio, Xiangyu Zhang, and Dongyan Xu. 2014. Face-change: Application-driven dynamic kernel view switching in a virtual machine. In *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. IEEE, 491–502.

[22] Steven A Hofmeyr, Stephanie Forrest, and Anil Somayaji. 1998. Intrusion detection using sequences of system calls. *Journal of computer security* 6, 3 (1998), 151–180.

[23] Christopher Jelesnianski, Mohannad Ismail, Yeongjin Jang, Dan Williams, and Changwoo Min. 2023. Protect the System Call, Protect (most of) the World with BASTION. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 528–541.

[24] Jinghao Jia, YiFei Zhu, Dan Williams, Andrea Arcangeli, Claudio Canella, Hubertus Franke, Tobin Feldman-Fitzthum, Dimitrios Skarlatos, Daniel Gruss, and Tianyin Xu. 2023. Programmable System Call Security with eBPF. *arXiv preprint arXiv:2302.10366* (2023).

[25] James C King. 1976. Symbolic execution and program testing. *Commun. ACM* 19, 7 (1976), 385–394.

[26] Anil Kurmus, Reinhard Tartler, Daniela Dorneanu, Bernhard Heinloth, Valentin Rothberg, Andreas Ruprecht, Wolfgang Schröder-Preikschat, Daniel Lohmann, and Rüdiger Kapitza. 2013. Attack Surface Metrics and Automated Compile-Time OS Kernel Tailoring.. In *NDSS*.

[27] Paul Lawrence. 2017. Seccomp filter in Android O. https://android-developers.googleblog.com/2017/07/seccomp-filter-in-android-o.html.

[28] Hugo Lefeuvre, Gaulthier Gain, Daniel Dinca, Alexander Jung, Simon Kuenzer, Vlad-Andrei Badoiu, Razvan Deaconescu, Laurent Mathy, Costin Raiciu, Pierre Olivier, et al. 2021. Unikraft and The Coming of Age of Unikernels. *login; The Usenix Magazine* (2021).

[29] Yihua Liao and V Rao Vemuri. 2002. Using text categorization techniques for intrusion detection.. In *USENIX Security Symposium*, Vol. 12. 51–59.

[30] Hong jiu Lu, Michael Matz, Milind Girkar, Jan Hubicka, Andreas Jaeger, and Mark Mitchell. 2018. System V Application Binary Interface – AMD64 Architecture Processor Supplement (With LP64 and ILP32 Programming Models). https://github.com/hjl-tools/x86-psABI/wiki/x86-64-psABI-1.0.pdf.

[31] AKM Fazla Mehrab, Ruslan Nikolaev, and Binoy Ravindran. 2022. Kite: lightweight critical service domains. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 384–401.

[32] Scott Michael. 2019. Oracle Linux Blog – What it means to be a maintainer of Linux seccomp. https://blogs.oracle.com/linux/being-a-seccomp-maintainer.

[33] Mozilla. 2016. Mozilla Wiki: Security/Sandbox/Seccomp. https://wiki.mozilla.org/Security/Sandbox/Seccomp.

[34] Darren Mutz, Fredrik Valeur, Giovanni Vigna, and Christopher Kruegel. 2006. Anomalous system call detection. *ACM Transactions on Information and System Security (TISSEC)* 9, 1 (2006), 61–93.

[35] Pierre Olivier, Daniel Chiba, Stefan Lankes, Changwoo Min, and Binoy Ravindran. 2019. A binary-compatible unikernel. In *Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 59–73.

[36] OpenSSH Contributors. 2022. OpenSSH Release Notes. https://www.openssh.com/releasenotes.html.

[37] Shankara Pailoor, Xinyu Wang, Hovav Shacham, and Isil Dillig. 2020. Automated policy synthesis for system call sandboxing. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–26.

[38] Chengbin Pang, Ruotong Yu, Yaohui Chen, Eric Koskinen, Georgios Portokalidis, Bing Mao, and Jun Xu. 2020. SoK: All You Ever Wanted to Know About x86/x64 Binary Disassembly But Were Afraid to Ask. *arXiv preprint arXiv:2007.14266* (2020).

[39] Qemu Contributors. 2022. Qemu Documentation: Security, Isolation Mechanisms. https://www.qemu.org/docs/master/system/security.html#isolation-mechanisms.

[40] Vidya Lakshmi Rajagopalan, Konstantinos Kleftogiorgos, Enes Göktas, Jun Xu, and Georgios Portokalidis. 2023. SysPart: Automated Temporal System Call Filtering for Binaries. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) *(CCS '23)*. Association for Computing Machinery, New York, NY, USA, 1979–1993. https://doi.org/10.1145/3576915.3623207

[41] Vaibhav Rastogi, Drew Davidson, Lorenzo De Carli, Somesh Jha, and Patrick McDaniel. 2017. Cimplifier: automatically debloating containers. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 476–486.

[42] Vaibhav Rastogi, Chaitra Niddodi, Sibin Mohan, and Somesh Jha. 2017. New directions for container debloating. In *Proceedings of the 2017 Workshop on Forming an Ecosystem Around Software Transformation*. 51–56.

[43] Paul Schaefer. 2016. VSFTPD: Bug in Seccomp Sandbox. https://bugs.archlinux.org/task/62889.

[44] David Schrammel, Samuel Weiser, Richard Sadek, and Stefan Mangard. 2022. Jenny: Securing Syscalls for PKU-based Memory Isolation Systems. In *USENIX Security Symposium*.

[45] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Audrey Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. 2016. SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis. In *IEEE Symposium on Security and Privacy*.

[46] Chia-Che Tsai, Bhushan Jain, Nafees Ahmed Abdul, and Donald E Porter. 2016. A study of modern Linux API usage and compatibility: what to support when you're supporting. In *Proceedings of the Eleventh European Conference on Computer Systems*. ACM, 16.

[47] David Wagner and R Dean. 2000. Intrusion detection via static analysis. In *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*. IEEE, 156–168.

[48] Zhiyuan Wan, David Lo, Xin Xia, and Liang Cai. 2019. Practical and effective sandboxing for Linux containers. *Empirical Software Engineering* 24, 6 (2019), 4034–4070.

[49] Zhiyuan Wan, David Lo, Xin Xia, Liang Cai, and Shanping Li. 2017. Mining sandboxes for linux containers. In *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 92–102.

[50] Ruoyu Wang, Yan Shoshitaishvili, Antonio Bianchi, Aravind Machiry, John Grosen, Paul Grosen, Christopher Kruegel, and Giovanni Vigna. 2017. Ramblr: Making Reassembly Great Again.. In *NDSS*.

[51] Christina Warrender, Stephanie Forrest, and Barak Pearlmutter. 1999. Detecting intrusions using system calls: Alternative data models. In *Proceedings of the 1999 IEEE symposium on security and privacy (Cat. No. 99CB36344)*. IEEE, 133–145.

[52] Richard Wartell, Yan Zhou, Kevin W Hamlen, Murat Kantarcioglu, and Bhavani Thuraisingham. 2011. Differentiating code from data in x86 binaries. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 522–536.

[53] Richard Wartell, Yan Zhou, Kevin W Hamlen, Murat Kantarcioglu, and Bhavani Thuraisingham. 2011. Differentiating code from data in x86 binaries. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2011, Athens, Greece, September 5-9, 2011, Proceedings, Part III 22*. Springer, 522–536.

[54] Andreas Wespi, Marc Dacier, and Hervé Debar. 2000. Intrusion detection using variable-length audit trail patterns. In *International Workshop on Recent Advances in Intrusion Detection*. Springer, 110–129.

[55] Jianliang Wu, Ruoyu Wu, Daniele Antonioli, Mathias Payer, Nils Ole Tippenhauer, Dongyan Xu, Dave Jing Tian, and Antonio Bianchi. 2021. LIGHTBLUE: Automatic Profile-Aware Debloating of Bluetooth Stacks. In *30th USENIX Security Symposium (USENIX Security 21)*. 339–356.